Studio Nx

Backend(FastAPI/Python) 38 — save_profile_image()

2026-10-12

Backend(FastAPI/Python) 38 — `save_profile_image()` (+ `image_extension()`)

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 프로필 사진 올리기 (PRD 01 FR-106, jpg/png/webp, 2MB)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	37 `update_me()` · 39 `delete_profile_image()`

1. 하는 일

올라온 사진을 검사한 다음 `storage/profiles/{무작위 32자}.{확장자}`에 저장하고, 회원의 `profile_image_path`를 바꿉니다. 예전 사진 파일은 지웁니다.

올라온 파일	결과
JPG (1.2MB)	`profiles/3f9c….jpg`로 저장되고, 응답의 사진 주소도 바뀝니다
브라우저는 `image/png`라고 알렸지만 내용은 JPG	저장합니다. 확장자는 **내용을 기준으로** `.jpg`가 됩니다
GIF	415 `UNSUPPORTED_FILE_TYPE` "Use a JPG, PNG or WebP image."
`image/png`라고 알렸지만 내용이 HTML이거나 빈 파일	똑같이 415입니다
2MB를 1바이트라도 넘음	413 `FILE_TOO_LARGE` "Images must be 2 MB or smaller."
정확히 2MB	저장합니다
이미 사진이 있음	새 사진을 저장한 뒤 예전 파일을 지웁니다

오류가 나면 파일이 하나도 남지 않고, DB도 그대로입니다.

2. 언제 쓰나

쓰는 곳	
`POST /users/me/profile-image` (42)	`multipart/form-data`의 `file` 칸을 받아 이 함수를 부르고, 36번으로 응답합니다
화면	userInfo.html의 Upload photo입니다 (P1-10). 화면도 같은 종류와 크기를 먼저 검사합니다 (`PHOTO_TYPES`, `PHOTO_MAX_BYTES`)

용량: 프로필 사진은 저장 한도 5GB의 사용량에 **넣지 않습니다** (2026-10-07 사용자 결정). 그래서 `storage_used_bytes`를 건드리지 않습니다.

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath
from uuid import uuid4
```

```
from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`uuid4`와 `get_settings`를 이번에 추가했습니다.

도우미 함수 (내용으로 파일 종류 알아내기)

```
def image_extension(data: bytes) -> str | None:
    if data.startswith(b"\xff\xd8\xff"):
        return ".jpg"
    if data.startswith(b"\x89PNG\r\n\x1a\n"):
        return ".png"
    if data[:4] == b"RIFF" and data[8:12] == b"WEBP":
        return ".webp"
    return None
```

함수

```
async def save_profile_image(db: Session, user: User, file: UploadFile) -> User:
    if file.content_type not in PHOTO_TYPES:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")
    data = await file.read(PHOTO_MAX_BYTES + 1)
    if len(data) > PHOTO_MAX_BYTES:
        raise AppError(ErrorCode.FILE_TOO_LARGE, 413, "Images must be 2 MB or smaller.")
    extension = image_extension(data)
    if extension is None:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")

    storage_dir = get_settings().storage_dir
    relative = f"{PROFILE_DIR}/{uuid4().hex}{extension}"
    new_file = storage_dir / relative
    new_file.parent.mkdir(parents=True, exist_ok=True)
    new_file.write_bytes(data)

    old_relative = user.profile_image_path
    user.profile_image_path = relative
    try:
        db.commit()
    except Exception:
        db.rollback()
        new_file.unlink(missing_ok=True)
        raise
    if old_relative:
        (storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from uuid import uuid4`	무작위 고유 번호를 만듭니다. `.hex`는 16진수 32자입니다
`from app.core.config import get_settings`	설정(`.env`)을 읽습니다. `storage_dir`는 `studionx/storage`(git에서 제외된 폴더)입니다

식	뜻
`async def`, `await file.read(n)`	파일을 **최대 n바이트까지만** 읽습니다. 업로드는 네트워크를 기다려야 하므로 FastAPI의 `UploadFile`은 비동기로 동작합니다
`b"\xff\xd8\xff"`	바이트 문자열입니다. `\xff`는 16진수 FF 한 바이트입니다
`data.startswith(…)`, `data[8:12]`	앞부분이 같은지 확인하고, 8번째부터 11번째 바이트를 꺼냅니다
`storage_dir / relative`	`Path`에서 `/`는 경로를 이어 붙입니다
`mkdir(parents=True, exist_ok=True)`	중간 폴더까지 만들고, 이미 있어도 오류를 내지 않습니다
`unlink(missing_ok=True)`	파일을 지우고, 파일이 없어도 오류를 내지 않습니다
`except Exception: … raise`	정리만 하고 **같은 오류를 다시** 던집니다 (오류를 삼키지 않습니다)

4. 흐름

1. 브라우저가 알려 준 종류를 확인하고, 허용하지 않는 종류면 415로 응답합니다.
2. 2MB + 1바이트까지만 읽고, 2MB를 넘으면 413으로 응답합니다.
3. 앞부분 바이트로 실제 종류를 확인하고, 아니면 415로 응답합니다. 확장자는 여기서 정합니다.
4. 무작위 이름으로 파일을 씁니다.
5. DB에 새 경로를 저장합니다. 실패하면 새 파일을 지우고 오류를 그대로 전달합니다.
6. 성공하면 예전 사진 파일을 지우고, DB에서 다시 읽어 돌려줍니다.

5. 왜 이렇게

선택	이유
종류를 **두 번** 확인 (알려 준 값과 실제 내용)	`content_type`은 브라우저(또는 공격자)가 마음대로 보낼 수 있습니다. HTML을 `image/png`라고 속여 저장시키면, 나중에 그 주소를 연 사람의 브라우저에서 스크립트가 실행될 위험이 있습니다
확장자를 **내용**으로 정함	파일을 돌려줄 때(40, 43) 확장자를 보고 종류를 정합니다. 그래서 내용과 확장자가 맞아야 합니다
`PHOTO_MAX_BYTES + 1`만 읽음	10GB짜리 파일을 올려도 2MB 남짓만 메모리에 올라옵니다. 1바이트를 더 읽어야 "딱 2MB"와 "2MB 초과"를 구분할 수 있습니다
저장 → DB → 예전 파일 삭제 순서 (템플릿은 "예전 파일 삭제 → 저장")	DB 저장이 실패해도 예전 사진이 남습니다. 템플릿 순서대로라면 실패했을 때 사진이 사라집니다. 시험으로 확인했습니다
이름은 무작위 32자 (회원 번호 없음)	35번의 사용자 결정입니다. 같은 회원이 사진을 바꾸면 이름도 바뀌어서 캐시 문제도 생기지 않습니다
`except Exception`으로 넓게 잡음	DB 연결이 끊기는 것처럼 어떤 오류가 나더라도 새 파일을 남기지 않기 위해서입니다. 오류는 다시 던져서 06번의 500 처리로 넘깁니다

선택	이유
그림 전체를 열어 보지는 않음	앞부분 바이트 검사로 종류는 확실히 가려낼 수 있습니다. 이미지 라이브러리(Pillow)로 다시 저장하면 더 안전하지만 패키지가 하나 늘어나므로, 필요해지면 나중에 추가합니다
사진 파일 응답에 `nosniff`	43번에서 `X-Content-Type-Options: nosniff`를 붙여서, 브라우저가 내용을 보고 종류를 바꿔 추측하지 않게 할 예정입니다

6. 시험 (`tests/test_user_service.py`에 17개를 추가해서 모두 39개, 실제 MySQL로 실행)

사진은 시험용 임시 폴더(`tmp_path`)에 저장했습니다. 실제 `storage/` 폴더는 건드리지 않았습니다.

시험	확인한 것
`image_extension` 7가지	jpg, png, webp는 인식하고, gif, HTML, WAV(RIFF이지만 WEBP가 아님), 빈 파일은 `None`입니다
JPG 저장	경로는 `profiles/{32자}.jpg`이고, 파일 내용이 같으며, 파일은 1개입니다. 응답 주소와 DB 경로가 바뀌고, `storage_used_bytes`는 0 그대로입니다
png라고 알렸지만 jpg	확장자가 `.jpg`입니다
GIF, HTML, 빈 파일, 2MB 초과	차례로 415, 415, 415, 413이고, 파일은 0개이며 경로는 그대로입니다
정확히 2MB	저장됩니다
사진 바꾸기	새 이름으로 저장되고, 예전 파일이 지워져서 파일은 1개입니다
예전 파일이 이미 없음	오류 없이 저장됩니다
DB 저장 실패	새 파일은 지워지고, 예전 사진 파일과 내용은 그대로이며, 오류가 그대로 전달됩니다

전체 시험 273개가 통과했고, `ruff` 검사도 통과했습니다.